Unit 4 - Python SQLite

Python SQLite3 module is used to integrate the SQLite database with Python. It provides a straightforward and simple-to-use interface for interacting with SQLite databases.

Installation

SQLite3 can be integrated with Python using sqlite3 module and it provides an SQL interface. You do not need to install this module separately because it is shipped by default along with Python version 2.5.x onwards.

To use sqlite3 module, you must first create a connection object that represents the database and then create a cursor object, which will help you in executing all the SQL statements.

SQLite Methods

Following are important sqlite3 module methods, which can suffice your requirement to work with SQLite database from your Python program.

1. **connect()**

To use SQLite3 in Python, first we have to import the *sqlite3* module and then create a connection object. Connection object allows to connect to the database and will let us execute the SQL statements.

Creating Connection object using the *connect()* function:

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
```

This will create a new file with the name 'mydatabase.db'.

2. **cursor()**

To execute SQLite statements in Python, we need a cursor object. We can create it using the *cursor()* method.

The SQLite3 cursor is a method of the connection object. To execute the SQLite3 statements, you should establish a connection at first and then create an object of the cursor using the connection object as follows:

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
```

Now we can use the cursor object to call the *execute()* method to execute any SQL queries.

3. **execute()**
   Once the database and connection object is created, we can create a table using CREATE TABLE statement. Then we execute the CREATE TABLE statement by calling cur.execute(...).

```
Eg:
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
cursorObj.execute('"CREATE TABLE movie(title text, year int, score real)"')
```

Here, 'movie' is the table name with columns title, year and score. Since SQLite is flexible, we can just use column names in the table declaration, specifying the data types is optional.

4. Close()

Once we are done with our database, it is a good practice to close the connection. We can close the connection by using the *close()* method.

To close a connection, use the connection object and call the *close()* method as follows:

```
con = sqlite3.connect('mydatabase.db')
#program statements
con.close()
```

SQLite database operations

I. Creating and connecting to Database

When you create a connection with SQLite, that will create a database file automatically if it doesn't already exist. This database file is created on disk with the connect function.

Following Python code shows how to connect to an existing database. If the database does not exist, then it will be created and finally a database object will be returned.

```
import sqlite3
conn = sqlite3.connect('test.db')
print ("Opened database successfully")
```

```
Output:
Opened database successfully
```

II. Create Table

To create a table in SQLite3, you can use the Create Table query in the *execute()* method. Consider the following steps:

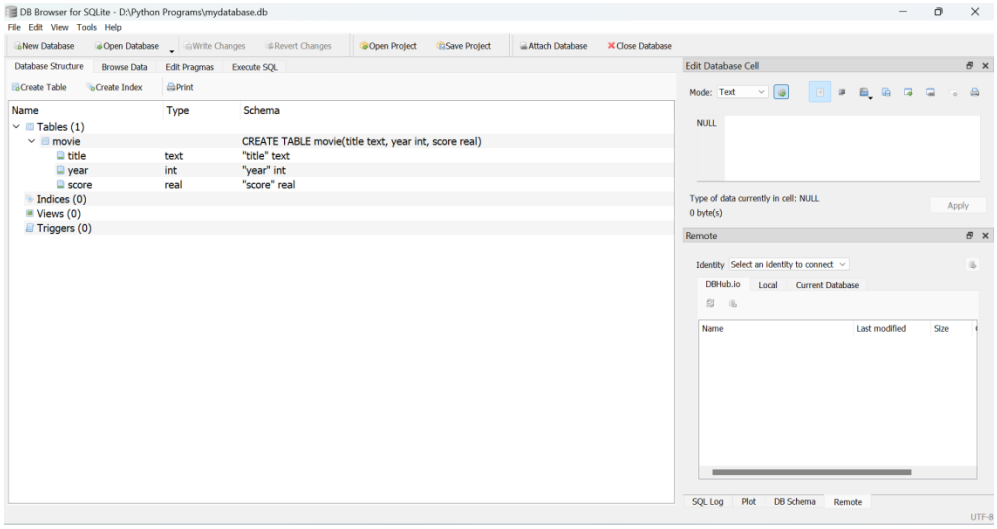
1. Create a connection object.
2. From the connection object, create a cursor object.
3. Using the cursor object, call the execute method with create table query as the parameter.

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
cursorObj.execute('''CREATE TABLE movie(title text, year int, score real)''')
con.commit()
con.close()
```

In the above code, it establishes a connection and creates a cursor object to execute the create table statement.

The *commit()* method saves all the changes we make.

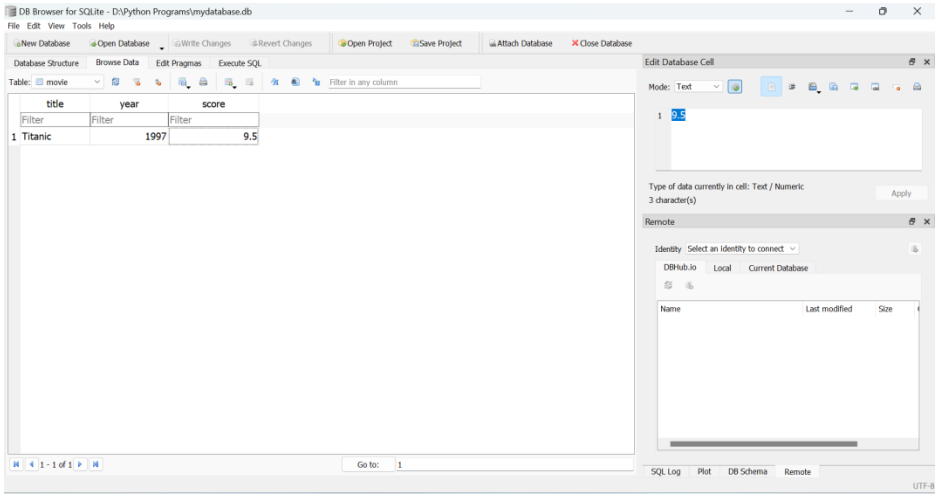
To check if our table is created, you can use the DB browser for SQLite(https://sqlitebrowser.org/dl/) to view your table. Open your mydatabase.db file with the program, and you should see your table:



III. Insert in Table

To insert data in a table, we use the INSERT INTO statement. Consider the following line of code:

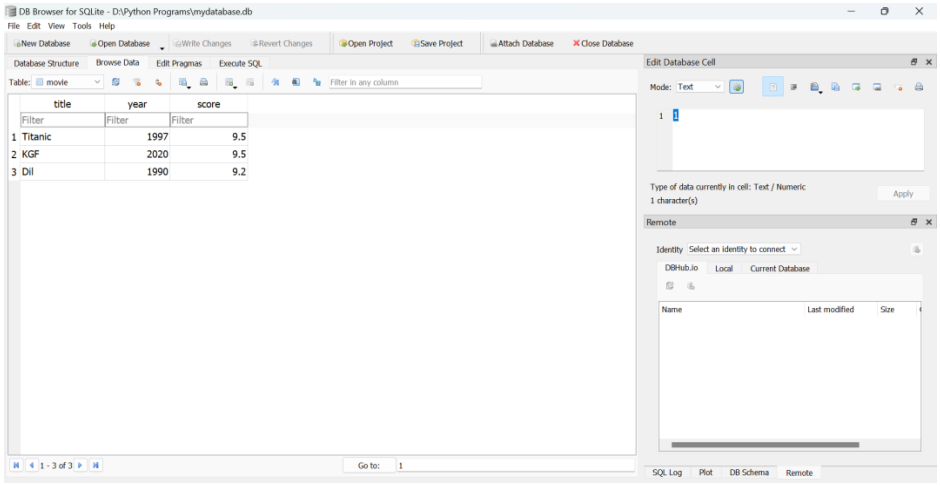
```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
cursorObj.execute('''CREATE TABLE movie(title text, year int, score real)''')
cursorObj.execute('''INSERT INTO movie VALUES  ("Titanic",1997, 9.5)''')
con.commit()
con.close()
```



We can also pass values to an INSERT statement in the *execute()* method. You can use the question mark (?) as a placeholder for each value. The syntax of the INSERT will be like the following:

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
m=input("Movie Name :")
y=int(input("Year :"))
s=float(input("Score :"))
cursorObj.execute('''INSERT INTO MOVIE VALUES (?,?,?) ''',(m,y,s))
con.commit()
con.close()
```

```
User Input:
Movie Name :Dil
Year :1990
Score :9.2
```



IV. Update Table

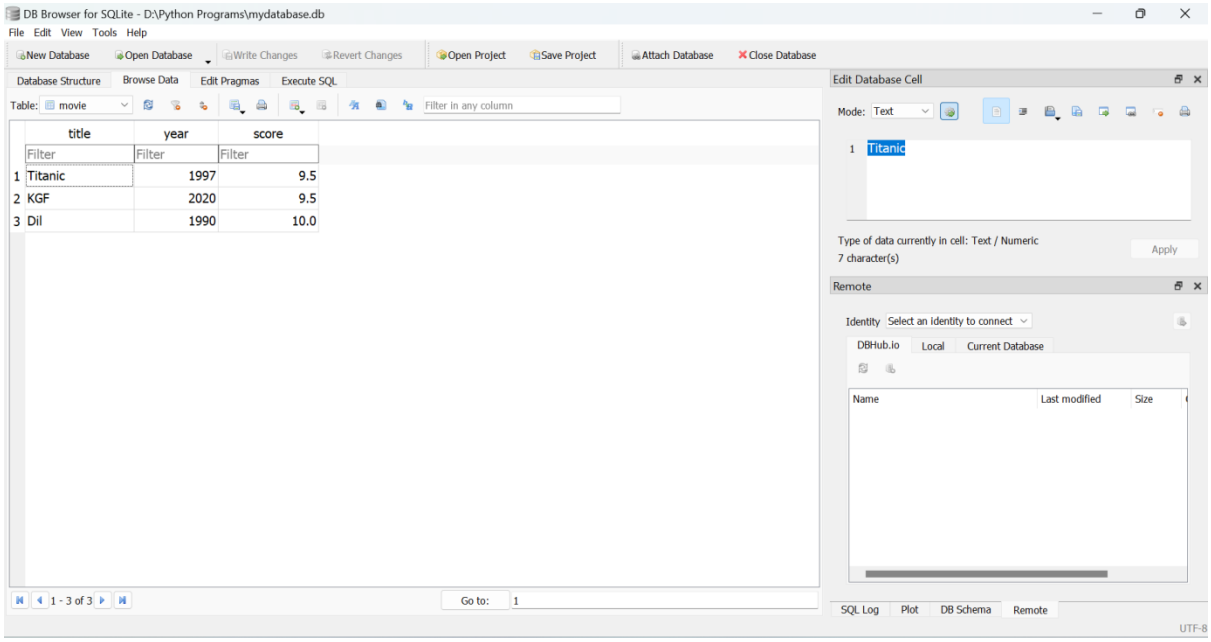
To update the table, simply create a connection, then create a cursor object using the connection and finally use the UPDATE statement in the *execute()* method.

Suppose that we want to update the score with the movie title Dil. For updating, we will use the UPDATE statement and for the movie whose title equals Dil. We will use the WHERE clause as a condition to select this employee.

Consider the following code:

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
cursorObj.execute("UPDATE MOVIE SET SCORE=10 WHERE TITLE='Dil' ")
con.commit()
con.close()
```

This will change the scorefor the movie Dil:



V. Select statement

You can use the select statement to select data from a particular table. If you want to select all the columns of the data from a table, you can use the asterisk (*). The syntax for this will be as follows:

```
select * from table_name
```

In SQLite3, the SELECT statement is executed in the execute method of the cursor object. For example, select all the columns of the movie table, run the following code:

```
cursorObj.execute("SELECT * FROM movie")
```

If you want to select a few columns from a table, then specify the columns like the following:

```
select column1, column2 from tables_name
```

For example,

```
cursorObj.execute( "SELECT title, year FROM movie")
```

The select statement selects the required data from the database table, and if you want to fetch the selected data, the *fetchall()* method of the cursor object is used. We will demonstrate this in the next section.

Fetch all data

To fetch the data from a database, we will execute the SELECT statement and then will use the *fetchall()* method of the cursor object to store the values into a variable. After that, we will loop through the variable and print all values.

The code will be like this:

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
cursorObj.execute("SELECT * FROM movie ")
records=cursorObj.fetchall()
for row in records:
    print(row)
con.commit()
con.close()
```

The above code will print out the records in our database as follows:

```
Output:
('Titanic', 1997, 9.5)
('KGF', 2020, 9.5)
('Dil', 1990, 10.0)
```

You can also use the *fetchall()* in one line as follows:

```
[print(row) for row in cursorObj.fetchall()]
```

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
cursorObj.execute("SELECT * FROM movie ")
[print(row) for row in cursorObj.fetchall()]
con.commit()
con.close()
```

If you want to fetch specific data from the database, you can use the WHERE clause.

VI. Delete

In SQLite database we use the following syntax to delete data from a table:

```
DELETE FROM table_name [WHERE Clause]
```

- Import the required module.
- Create a connection object with the database using to connect().
- Create a Cursor object by calling the cursor().
- Finally, use execute() method bypassing a DELETE statement as a parameter to it.

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
#Deleting
cursorObj.execute("Delete from movie where title='Titanic' ")
con.commit()
con.close()
```

VII. Drop table

You can drop/delete a table using the DROP statement. The syntax of the DROP statement is as follows:

```
drop table table_name
```

To drop a table, the table should exist in the database. Therefore, it is recommended to use "*if exists"* with the drop statement as follows:

```
drop table if exists table_name
```

For example,

```
import sqlite3
con = sqlite3.connect('mydatabase.db')
cursorObj = con.cursor()
cursorObj.execute("DROP TABLE IF EXISTS MOVIE ")
con.commit()
con.close()
```

#Program to demonstrate a database operations- create, insert, select, update, delete and drop

```
#importing the module
import sqlite3

#create connection object
con = sqlite3.connect('mydatabase.db')

#crete a cursor
cursorObj = con.cursor()

#creating the table
cursorObj.execute('''CREATE TABLE movie(title text, year int, score real)''')

#inserting the data
cursorObj.execute('''INSERT INTO movie VALUES  ("Titanic",1997, 9.5)''')
cursorObj.execute('''INSERT INTO movie VALUES  ("KGF",2020, 9.1)''')
cursorObj.execute('''INSERT INTO movie VALUES  ("Dil",1990, 8.5)''')

#Print the Initial Data
print("Initial Data...")
cursorObj.execute("SELECT * FROM movie ")
[print(row) for row in cursorObj.fetchall()]

#Updating
cursorObj.execute("UPDATE MOVIE SET SCORE=10 WHERE TITLE='Dil' ")

#Print the after updating
print("After updating...")
cursorObj.execute("SELECT * FROM movie ")
[print(row) for row in cursorObj.fetchall()]
```

```
#Deleting
cursorObj.execute("Delete from movie where title='Titanic' ")

#Print the after Deleting
print("After deleting...")
cursorObj.execute("SELECT * FROM movie ")
[print(row) for row in cursorObj.fetchall()]

#Drop the table
cursorObj.execute(" DROP TABLE IF EXISTS MOVIE ")

#commit changes in the database
con.commit()

#close the connection
con.close()
```

```
Output
Initial Data...
('Titanic', 1997, 9.5)
('KGF', 2020, 9.1)
('Dil', 1990, 8.5)

After updating...
('Titanic', 1997, 9.5)
('KGF', 2020, 9.1)
('Dil', 1990, 10.0)

After deleting...
('KGF', 2020, 9.1)
('Dil', 1990, 10.0)
```

NumPy in Python

NumPy is the fundamental package for scientific computing with Python. It stands for "Numerical Python." It is used for:

- N-dimensional array object
- Broadcasting functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

NumPy's array class is called ndarray. In NumPy arrays, the individual data items are called elements. All elements of an array should be of the same type. Arrays can be made up of any number of dimensions. In NumPy, dimensions are called axes. Each dimension of an array has a length which is the total number of elements in that direction. The size of an array is the total number of elements contained in an array in all the dimension. The size of NumPy arrays are fixed; once created it cannot be changed again.

All the rows are indicated by Axis-0 and all the columns are indicated by Axis-1. Array indexes are 0-based. That is, if the length of a dimension is n, the index values range from 0 to n – 1.

In order to use NumPy in your program, you need to import NumPy. For example,

import numpy as np

numpy is usually renamed as np.

NumPy Arrays Creation Using array() Function

We can create a NumPy array from a regular Python list or tuple using the np.array() function. The type of the resulting array is deduced from the type of the elements.

For example,

Example	Output

`importnumpyasnp`
`num1=np.array([5,6,7,4,3,2])`
`print(num1)`
`print(type(num1))`
	[5 6 7 4 3 2]
	<class 'numpy.ndarray'>

`importnumpyasnp`
`num2=np.array([5.5,6.6,7.7,8.8])`
`print(num2)`
`print(type(num2))`
	[5.5 6.6 7.7 8.8]
	<class 'numpy.ndarray'>

`#creating two dimentional array`
`importnumpyasnp`
`twoD=np.array(([1,2,3],[4,5,6]))`
`print(twoD)`
`print(type(twoD))`
	[[1 2 3]
	[4 5 6]]
	<class 'numpy.ndarray'>

`#creating two dimentional array`
`importnumpyasnp`
`twoD1=np.array(((1,2,3),(4,5,6)))`
`print(twoD1)`
`print(type(twoD1))`
	[[1 2 3]
	[4 5 6]]
	<class 'numpy.ndarray'>

Array Attributes

The contents of ndarray can be accessed and modified by indexing or slicing the array and via the methods and attributes of the ndarray. The more important attributes of ndarray object are:

Array Attributes

ndarray Attributes	Description
ndarray.ndim	Gives the number of axes or dimensions in the array
ndarray.shape	Gives the dimensions of the array. For an array with n rows and m columns, shape will be a tuple of integers (n, m).
ndarray.size	Gives the total number of elements of the array.
ndarray.dtype	Gives an object describing the type of the elements in the array. One can create or specify dtype's using standard Python types. Additionally, NumPy provides its own types like np.int32, np.int16, np.float64, and others.
ndarray.itemsize	Gives the size of each element of the array in bytes.
ndarray.data	Gives the buffer containing the actual elements of the array. Normally, we will not use this attribute because we will access the elements in an array using indexing facilities.

Note: In your code, replace *ndarray* with you Python NumPy ndarray object name.

Example:

```
importnumpyasnp
a1=np.array(([1,2,3],[4,5,6]))
print(a1.ndim)      # output 2
print(a1.shape)     # output  (2,3)
print(a1.size)      # output : 6
print(a1.dtype)     # output : dtype('int32)
print(a1.itemsize)  # output : 4
print(a1.data)      # output : <memory at 0x0000014DB7F518A0>
```

NumPy Arrays Creation with Initial Placeholder Content

Often, the elements of an array are initially unknown, but its size is known. Hence, NumPy offers several functions to create arrays with initial placeholder content. These minimize the necessity of growing arrays, an expensive operation.

1. **np.zeros()** is used to create an array filled with zeros.

```
importnumpyasnp
arr1=np.zeros(5)
print(arr1)  # Output: [0. 0. 0. 0. 0.]
```

2. **np.ones()** is used to create an array filled with ones.

```
importnumpyasnp
arr1=np.ones(5)
print(arr1)  # Output: [1. 1. 1. 1. 1.]
```

4. **np.empty()** is used to create an array without initializing its elements to any particular value.

```
importnumpyasnp
arr=np.empty(5)
print(arr)
# output: [1.09813107e-311 4.94065646e-324 0.00000000e+000
5.09278990e-313 8.70018274e-313]
```

5. **np.full()** is used to create an array filled with a specified value.

```
importnumpyasnp
arr=np.full(5,2)
print(arr)            #output : [2 2 2 2 2]
```

6. **np.eye()** is used to create an identity matrix.

```
importnumpyasnp
arr=np.eye(2)
print(arr)
```

7. **np.random.random()** is used to generate random numbers from a uniform distribution over the range [0, 1).

```
importnumpyasnp
# Generate a single random value
value=np.random.random()
print(value)  # Output: a random value between 0 and 1
```

```
importnumpyasnp
# Generate a 1D array of random values
value=np.random.random(3)
print(value)  # Output: an array of 3 random values between 0 and 1
```

8. **np.arange()** is used to create an array of evenly spaced values within a specified range.
   The syntax for arange() is, **np.arange([start,]stop, [step,][dtype=None])**
   Returns evenly spaced values within a given interval where start (a number and optional) is the start of interval and its default value is zero, stop (a number) is the end of interval, and step (a number and is optional) is the **spacing between the values** and dtype is the type of output array.

```
importnumpyasnp
arr=np.arange(0, 2, 0.3)
print(arr) #output: [0. , 0.3, 0.6, 0.9, 1.2, 1.5, 1.8]
```

9. **np.linspace()** is used to create an array of evenly spaced values within a specified interval.
   The syntax for linspace is, **numpy.linspace(start, stop, num=50, dtype=None)**
   Returns evenly spaced numbers over a specified interval where start is the starting value of the sequence, stop is the end value of the sequence, and num (an integer and optional)

is the **number of samples to generate**. Default is 50. Must be non-negative. The optional dtype is the type of the output array.

```
importnumpyasnp
arr=np.linspace(0, 2, 5)
print(arr) #output: [0.  0.5 1.  1.5 2. ]
```

Integer Indexing, Array Indexing, Boolean Array Indexing, Slicing and Iterating in Arrays

One-dimensional arrays can be indexed, sliced, and iterated over, much like lists and other Python sequences

Indexing, slicing, and iterating operations on one-dimensional NumPy arrays

```
#Indexing and slicing
importnumpyasnp
arr=np.arange(5)
print(arr[2])     #output: 2
print(arr[2:4])   #output: [2 3]
arr[:4:2]=8
print(arr)        #output: [8 1 8 3 4]
print(arr[::-1])  #output: [4 3 8 1 8]
#Iteration
foreleinarr:
    print(ele)
      #output
      8
      1
      8
      3
      4
```

For multi-dimensional arrays you can specify an index or slice per axis.

```
#Indexing and slicing
importnumpyasnp
a=np.array([[1,2,3],[4,5,6],[6,7,8]])
print(a)
      '''
      output :
      [[1 2 3]
      [4 5 6]
      [6 7 8]]
      '''
print(a[1,2])  #Output: 6
print(a[:2,1:3])
      '''
      output:
      [[2 3]
       [5 6]]
      '''
print(a[1,:])  #lower axis output: [4 5 6]
```

```
print(a[:,1])  #output: [2 5 7]
#Iteration over first axis
forrowina:
    print(row)
            '''
            output:
            [1 2 3]
            [4 5 6]
            [6 7 8]
            '''
#Iteration over each element using flat attribute
foreleina.flat:
    print(ele)
            '''
            output:
            1
            2
            3
            4
            5
            6
            6
            7
            8
            '''
```

Basic Arithmetic Operations on NumPy Arrays

Basic mathematical functions perform element-wise operation on arrays and are availableboth as operator overloads and as functions in the NumPy module.

Arithmetic operations on one-dimensional array

```
importnumpyasnp
a=np.array([1,2,3,4])
b=np.arange(4)          #[0 1 2 3]
print(a+b)              #[1 3 5 7]
print(np.add(a,b))      #[1 3 5 7]
print(a-b)              #[1 1 1 1]
print(np.subtract(a,b)) #[1 1 1 1]
print(a*b)              #[0 2 6 12]
print(np.divide(a,b))   #[inf  2.  1.5  1.33333333]
```

Arithmetic operations on two-dimensional array

```
importnumpyasnp
A=np.array( [[1, 2], [3, 4]] )
B=np.array( [[2, 3], [4, 5]] )
print(A)
            '''
            [[1 2]
             [3 4]]
            '''
```

```
print(B)
            '''
            [[2 3]
             [4 5]]
            '''
print(A+B)
            '''
            [[3 5]
             [7 9]]
            '''
print(A-B)
            '''
            [[-1 -1]
             [-1 -1]]
            '''
print(np.multiply(A,B))
            '''
            [[ 2  6]
             [12 20]]
            '''
print(np.dot(A,B))
            '''
            [[10 13]
             [22 29]]
            '''
print(B*2)
            '''
            [[ 4  6]
             [ 8 10]]
            '''
```

Pandas

Pandas library is a Python library that provides fast, flexible, and expressive data structuresdesigned to make working with "relational" or "labeled" data both easy and intuitive.

The two primary data structures of pandas, Series (one-dimensional) and DataFrame(two-dimensional), handle the vast majority of typical-use cases in finance, statistics,socialscience, and many areas of engineering. pandas is built on top of NumPy and isintendedto integrate well within a scientific computing environment with many other third-partylibraries.

pandas is well suited for inserting and deleting columns from DataFrame, for easy handling of missing data (represented as NaN), explicitly aligning data to a set of labels,converting data in other Python and NumPy data structures into DataFrame objects,intelligent label-based slicing, indexing, and subsetting of large data sets, merging andjoining of data sets, and flexible reshaping. Additionally, it has robust input/output toolsfor loading data from CSV files, Excel files, databases, and other formats. You have toimport a pandas library to make use of various functions and data structures defined inpandas.

import pandas as pd

pandas is usually renamed as pd.

Pandas Series

Series is a one-dimensional labeled array capable of holding any data type (integers, strings, floating point numbers, Python objects, etc.). The axis labels are collectively referred to as the index. Pandas Series is created using series() method and its syntax is,

s = pd.Series(data, index=None)

Here, s is the Pandas Series, data can be a Python dict, a ndarray, or a scalar value (like 5).

The passed index is a list of axis labels. Both integer and label-based indexing are supported. If the index is not provided, then the index will default to range(n) where n is the length of data. For example,

i. Create Series from ndarrays

To create a Pandas Series from ndarrays (NumPy arrays), you can use the pd.Series() function.

```
importnumpyasnp
importpandasaspd
# Create ndarrays
data=np.array([10, 20, 30, 40, 50])
index=np.array(['A', 'B', 'C', 'D', 'E'])
# Create Series from ndarrays
```

```
output
A    10
B    20
C    30
D    40
E    50
```

```
series=pd.Series(data, index=index)
# Print the Series
print(series)
```

```
dtype: int32
```

ii. Create Series from Dictionaries

pd.Series() function is used to create dictionaries. The keys of the dictionary will become the index labels of the Series, and the corresponding values will become the data in the Series.

```
importpandasaspd
# Create a dictionary
data= {'A': 10, 'B': 20, 'C': 30, 'D': 40, 'E': 50}
# Create Series from dictionary
series=pd.Series(data)
# Print the Series
print(series)
```

```
output
A    10
B    20
C    30
D    40
E    50
dtype: int64
```

iii. Create Series from Scalar data

To create a Pandas Series from scalar data, you can use the pd.Series() function and provide the scalar value along with the desired index labels.

```
importpandasaspd
# Create a scalar value
scalar_value=5
# Create Series from scalar value
series=pd.Series(scalar_value, index=['A', 'B',
'C', 'D', 'E'])
# Print the Series
print(series)
```

```
output
A    5
B    5
C    5
D    5
E    5
dtype: int64
```

iv. Series Indexing and Slicing

In Pandas, indexing and slicing are common operations used to access and manipulate data within a Series. Series indexing allows you to retrieve specific elements based on their position or index label, while slicing allows you to extract a subset of elements from the Series.

```
importpandasaspd

# Create a Series
series=pd.Series([10,20,30,40,50], index=['A','B','C','D','E'])

# Indexing by position
print(series[0])    # Output: 10
print(series[2])    # Output: 30
```

```
# Indexing by label
print(series['A'])  # Output: 10
print(series['C'])  # Output: 30




# Slicing by position
print(series[1:4])  # Output: B    20
                    #          C    30
                    #          D    40
                    #          dtype: int64

# Slicing by label
print(series['B':'D'])  # Output: B    20
                        #          C    30
                        #          D    40
                        #          dtype: int64
```

v. Working with Text Data

The Pandas Series supports a set of **string processing methods** that make it easy to operateon each element of the array. These methods are accessible via the str attribute and theygenerally have the same name as that of the built-in Python string methods.

```
importnumpyasnp
importpandasaspd
states=pd.Series(["Karnataka", "Kerala", "Kolkatha", "Haryana"])

print(states.str.lower())
            '''
            0    karnataka
            1       kerala
            2     kolkatha
            3      haryana
            dtype: object
            '''
print(states.str.upper())
            '''
            0    KARNATAKA
            1       KERALA
            2     KOLKATHA
            3      HARYANA
            dtype: object
            '''
print(states.str.len())
            '''
            0    9
            1    6
            2    8
```

```
            3    7
            dtype: int64
            '''




print(states.str.count('a'))
                '''
                0    4
                1    2
                2    2
                3    3
                dtype: int64
                '''
print(states.str.startswith('K'))
                '''
                dtype: int64
                0     True
                1     True
                2     True
                3    False
                dtype: bool
                '''
print(states.str.replace('Kerala','Tamilnadu'))
                '''
                0    Karnataka
                1    Tamilnadu
                2     Kolkatha
                3      Haryana
                dtype: object
                '''
```

Pandas DataFrame

DataFrame is a two-dimensional, labeled data structure with columns of potentially different types. DataFrame accepts many different kinds of input like Dict of one-dimensional ndarrays, lists, dicts, or Series, two-dimensional ndarrays, structured or record ndarray, a dictionary of Series, or another DataFrame.

df = pd.DataFrame(data=None, index=None, columns=None)

Here, df is the DataFrame and data can be NumPy ndarray, dict, or DataFrame. Along with the data, you can optionally pass an index (row labels) and columns (column labels) attributes as arguments. If you pass an index and/or columns, you are guaranteeing the index and/or columns of the resulting DataFrame. Both index and columns will default to range(n) where n

is the length of data, if they are not provided. When the data is a dictionary and columns are not specified, then the DataFrame column labels will be dictionary's keys.

i. Create DataFrame from Dictionary of Series/Dictionaries

a. DataFrame from Dictionary of Series:

```
importpandasaspd
# Dictionary of Series
data= {
'Name': pd.Series(['John', 'Alice', 'Bob']),
  'Age': pd.Series([25, 28, 30]),
}
# Create DataFrame from the dictionary
df=pd.DataFrame(data)
# Print the DataFrame
print(df)
```

```
    Name  Age
0   John   25
1  Alice   28
2    Bob   30
```

b. DataFrame from Dictionary of Dictionaries:

```
importpandasaspd
# Dictionary of dictionaries
data= {
    'Name': {'A': 'John', 'B': 'Alice', 'C': 'Bob'},
    'Age': {'A': 25, 'B': 28, 'C': 30},
}
# Create DataFrame from the dictionary
df=pd.DataFrame(data)
# Print the DataFrame
print(df)
```

```
    Name  Age
0   John   25
1  Alice   28
2    Bob   30
```

ii. Create DataFrame from ndarrays/lists/list of dictionaries

a. Creating a DataFrame from ndarrays:

```
import pandas as pd
import numpy as np
data = np.array([[1, 2, 3], [4, 5, 6]])  #ndarray
df = pd.DataFrame(data, columns=['A', 'B', 'C'])
print(df)
```

```
   A  B  C
0  1  2  3
1  4  5  6
```

b. Creating a DataFrame from lists:

```
import pandas as pd
data = [['Alice', 25], ['Bob', 30], ['Charlie', 35]]  #list
df = pd.DataFrame(data, columns=['Name', 'Age'])
print(df)
```

```
      Name  Age
0    Alice   25
1      Bob   30
2  Charlie   35
```

c. Creating a DataFrame from a list of dictionaries:

```
import pandas as pd
# Create a list of dictionaries
data = [
    {'Name': 'Alice', 'Age': 25},
    {'Name': 'Bob', 'Age': 30},
    {'Name': 'Charlie', 'Age': 35}
]
df = pd.DataFrame(data)
print(df)
```

```
   Name  Age
0  Alice   25
1    Bob   30
2  Charlie   35
```

iii. Creating Data Frame from Python list of Tuples

To create a DataFrame from a list of tuples in Python using pandas, you can use the pd.DataFrame() constructor. Each tuple in the list represents a row of data, and the elements within each tuple correspond to the values in each column of the DataFrame.

```
import pandas as pd
# Create a list of tuples
data = [
    ('Alice', 25),
    ('Bob', 30),
    ('Charlie', 35)
]
df = pd.DataFrame(data, columns=['Name', 'Age'])
print(df)
```

```
      Name  Age
0    Alice   25
1      Bob   30
2  Charlie   35
```

iv. Creating Data frame from an Excel spreadsheet

To create a DataFrame from an Excel file in Python using pandas, you can use the **pd.read_excel()** function. This function reads data from an Excel file and returns a DataFrame. Here's an example:

```
import pandas as pd
# Read Excel file into a DataFrame
df = pd.read_excel('data.xlsx')
# Display DataFrame
print(df)
```

v. Creating Data Frame from .csv files

A CSV (Comma-Separated Values) file is a plain text file format used to store tabular data, such as a spreadsheet or a database table.In a CSV file, each line represents a row of data, and the values within a row are separated by commas. It is a simple and widely supported file format that is easy to create, read, and manipulate.

To create a DataFrame from a CSV file in Python using pandas, you can use the **pd.read_csv()** function. This function reads data from a CSV file and returns a DataFrame.

```
import pandas as pd
# Read CSV file into a DataFrame
df = pd.read_csv('data.csv')
# Display DataFrame
print(df)
```

Operations on DataFrame

Consider the dataframe

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({'Name': ['Alice', 'Bob', 'Charlie'],
                   'Age': [25, 30, 35],
                   'Gender': ['Female', 'Male', 'Male']})
print(df)
```

```
      Name  Age  Gender
0    Alice   25  Female
1      Bob   30    Male
2  Charlie   35    Male
```

i. Knowing the number of rows and columns

```
print(df.shape)
```

```
(3, 3)
```

ii. Retrieving Rows from data frame / Displaying data in Dataframe

The head() method gives first 5 rows and tail() returns last 5 rows in the dataframe.

```
print(df.head())
```

```
print(df.tail())
```

```
# display first 2 rows
print(df.head(2))
```

```
# display last 2 rows
print(df.tail(2))
```

iii. Retrieving a range of Rows

Retrieving multiple rows from a dataframe is done using slicing. For example, df[2:5] returns 2nd, 3rd and 4th row.

```
print(df[2:5])
```

```
# display alternate rows
print(df[0::2])
```

```
#display rows in reverse order
print(df[5:0:-1])
```

iv. Column selection / Retrieving column data

We use column name as subscript to retrieve the column.

For multiple column data, we provide list of subscript.

df.columns returns the names of the columns from the dataframe.

```
# Select a single column
name_column = df['Name']
# or name_column = df.Name
print(name_column)

# Select multiple columns
subset = df[['Name', 'Age']]
print(subset)
```

```
0      Alice
1        Bob
2    Charlie
Name: Name, dtype: object

      Name  Age
0    Alice   25
1      Bob   30
2  Charlie   35
```

v. Addition and Deletion of columns

Adding column

```
import pandas as pd
# Create a DataFrame
df = pd.DataFrame({'Name': ['Alice', 'Bob', 'Charlie'],
                   'Age': [25, 30, 35],
                   'Gender': ['Female', 'Male', 'Male']})

# Add a new column using subscript
df['City'] = ['New York', 'London', 'Paris']
print(df)
```

```
     Name  Age  Gender      City
0   Alice   25  Female  New York
1     Bob   30    Male    London
2 Charlie   35    Male     Paris
```

```
#adding column using insert()
df.insert(2,'class',['I','II','III'])
print(df)
```

```
     Name  Age class  Gender      City
0   Alice   25     I  Female  New York
1     Bob   30    II    Male    London
2 Charlie   35   III    Male     Paris
```

```
#deleting a column using column name subscript
del df['Gender']
print(df)
```

```
     Name  Age class      City
0   Alice   25     I  New York
1     Bob   30    II    London
2 Charlie   35   III     Paris
```

```
#deleting column using pop() function
df.pop('class')
print(df)
```

```
     Name  Age      City
0   Alice   25  New York
1     Bob   30    London
2 Charlie   35     Paris
```

vi. Finding minimum and maximum values

Highest value can be found is using max() method and least value can be found using min() method. These methods can be applied on columns containing numerical data.

```
print(df['Age'].max())
output: 35
```

```
print(df['Age'].min())
output: 25
```

vii. Displaying statistical information

The describe() method is used to display the informations like the number of values, average, standard deviation, minimum, maximum, 25%, 50% and 75% of the total value.

print(df.describe())

output:

```
Age
count   3.0
mean   30.0
std     5.0
min    25.0
25%    27.5
50%    30.0
```

```
75%   32.5
max   35.0
```

viii. Performing Queries on Data

We can retrieve rows based on query. The query should be given as subscript in the data frame object.

`print(df[df.Age>25])`	Name Age Gender 1 Bob 30 Male 2 Charlie 35 Male
`print(df[df.Age==df.Age.max()])`	Name Age Gender 2 Charlie 35 Male

Data Visualization

Representing the data in the form of pictures or graphs is called 'data visualization'. It becomes easy to understand the data through the pictures. Python provides several libraries for data visualization, with the most popular ones being Matplotlib, Seaborn, and Plotly.

Matplotlib is a plotting library for creating static, animated, and interactive visualizations in Python.

Installing Matplotlib : pip install matplotlib

Pyplot is a sub-module of the matplotlib library for Python, which contains a collection of functions/methods used for plotting simple 2D graphs using Python.

Different Types of Charts using Pyplot

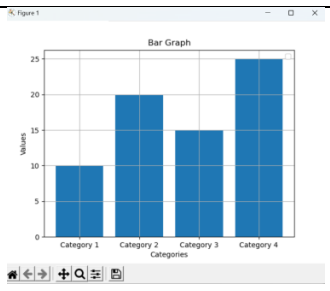
Bar Graph

A bar graph represents the data in the form of vertical or horizontal bars. It is used to compare the quantities.

Steps to create a bar graph:

1. Import the modules
2. Prepare your data:
   a. Define the categories or labels for the x-axis.
   b. Create a list of values for the corresponding y-axis.
3. Create the bar graph:
   a. Use the bar() function from pyplot module to create the bar graph.
   b. Pass the x-axis categories and y-axis values as arguments.
4. Customize the graph (optional):
   - You can customize various aspects of the bar graph, such as the colors, labels, title, axis labels, and grid lines.
   - #legend refers to colors and labels of data when different sets of data are plotted
5. Display the graph:
   - Use the show() function to display the bar graph.

```
#import modules
importmatplotlib.pyplotasplt
#prepare data
categories= ['Category 1', 'Category 2', 'Category 3', 'Category 4']
values= [10, 20, 15, 25]
#draw the bar graph
plt.bar(categories, values)
#labels on x and y axes
plt.xlabel('Categories')
plt.ylabel('Values')
#Title of the bar graph
plt.title('Bar Graph')
plt.grid(True)   #draw grid
plt.legend()
plt.show()
```

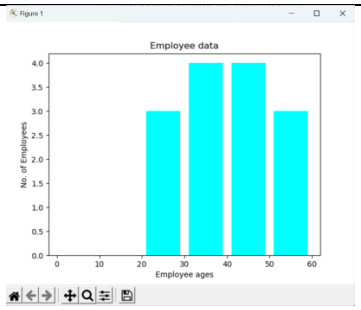


Histogram

Histogram shows distribution of values grouped in bins or intervals. Histograms are commonly used to visualize the frequency distribution of continuous or discrete numerical data. hist() function is used to create histograms. The hist() function will use an array of numbers to create a histogram.

A program to display histogram

```
#histogram
importmatplotlib.pyplotasplt
emp_ages=[20,30,25,35,60,45,55,43,46,27,66,51,49,33,38]
bins=[0,10,20,30,40,50,60]
plt.hist(emp_ages,bins,histtype='bar',rwidth=0.8,color='cyan')
plt.xlabel('Employee ages')
plt.ylabel('No. of Employees')
plt.title('Employee data')
plt.show()
```



Pie Chart

A pie chart shows a circle that is divided into sectors and each sector represents a proportion of the whole. pie() function is used to draw the pie chart.

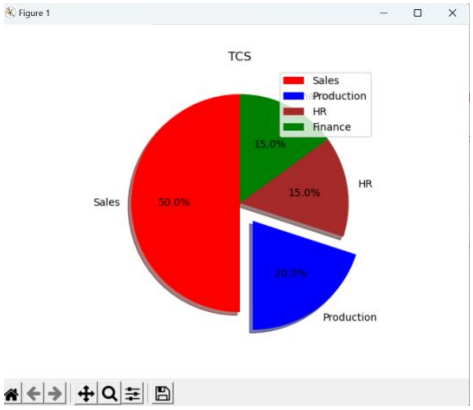
Syntax:

plt.pie(data, explode=None, labels=None, colors=None, autopct=None, shadow=False)

data represents the array of data values to be plotted
labels is a list of sequence of strings which sets the label
color attribute is used to provide color the slice
autopct is a string used to label the slice with their numerical value.
shadow is used to create shadow of slice
explode indicates whether the slice of the pie should stand out or not.

A Python program to display Pie Chart showing percentage of employees in each department. Assume there are 4 departments namely Sales , Production , HR and Finance.

```
#pie chart
importmatplotlib.pyplotasplt
slices=[50,20,15,15]
depts=['Sales','Production','HR','Finance']
cols=['red','blue','brown','green']
plt.pie(slices,labels=depts, colors=cols, startangle=90,
explode=(0,0.2,0,0),shadow=True, autopct='%.1f%%')
plt.title('TCS')
plt.legend()
plt.show()
```



Line Graph

A line graph is a simple graph that shows the results in the form of lines. To create a line graph, we need x and y coordinates.

Syntax: plt.plot(x,y,'colorname)

A Python Program to create Line Graph showing number of students of a college in various Years. Consider 8 years data.

```
#Line graph
importmatplotlib.pyplotasplt
years=['2012','2013','2014','2015','2016','2017','2018','2019']
students=[30,40,10,60,30,45,55,25]
plt.plot(years,students,'red')
plt.title("Student strength")
plt.xlabel('Year')
plt.ylabel('No. of students')
plt.show()
```

